

Manual de Usuario — Python Logic Checker

Analisis en linea o local



Extensión para Visual Studio Code que detecta errores lógicos en Python usando Inteligencia Artificial.

*Desarrollada para el **Diplomado en Metodologías Ágiles y Tecnologías Avanzadas para el Desarrollo de Software** — Universidad de Nariño.*

¿Para qué sirve esta extensión?

Cuando aprendemos a programar en Python, no todos los errores son de escritura. Algunos programas se ejecutan sin problema, pero producen resultados incorrectos. Esos son los **errores lógicos**, y son los más difíciles de encontrar. **Python Logic Checker** analiza tu código automáticamente cada vez que guardas el archivo y te muestra, directamente en el editor, qué líneas tienen problemas y cómo puedes solucionarlos.

Requisitos previos

Hardware

⚠ **Nota : Requisitos previos para la extensión con funcionamiento local**

Antes de instalar la extensión necesitas tener lo siguiente en tu computador:

Requisitos mínimos.

- **RAM:** 8 GB (mínimo absoluto), aunque se recomiendan 16 GB para un mejor rendimiento general del sistema. **El modelo phi3:** mini por sí solo suele ocupar ~3 GB de RAM durante la ejecución.
- **Almacenamiento:** 4 GB a 10 GB de espacio libre en disco (para Ollama y el modelo).
- **Procesador (CPU):** Procesador de 64 bits con soporte AVX2 (la mayoría de las CPUs desde 2013 en adelante). Un procesador de 4 núcleos es lo mínimo recomendado.
- **Gráficos (GPU) - Opcional:** No es necesario tener GPU dedicada. Ollama puede ejecutar phi3 solo con la CPU, aunque la GPU acelera la velocidad de generación.
- **Sistema Operativo:** Windows 10/11, macOS 12+ o Linux (Ubuntu 22.04+ recomendado).

Software

- **Visual Studio Code:** versión 1.90 o superior. Puedes descargarlo gratis en code.visualstudio.com.
- **API Key de OpenRouter:** una clave que permite a la extensión usar IA. Más abajo encontrarás los pasos para obtenerla.

⚠ **Nota : Requisitos previos para la extensión con funcionamiento local**

- **Ollama:** para análisis local y privacidad
- **Modelo :** phi3

Instalación

Opción A — Instalar desde el archivo .vsix

1. Ingresa al repositorio <https://github.com/jhonatanjcdg/Python-Logic-Checker/blob/develop/python-logic-checker-1.0.1.vsix>
2. Descarga el archivo en tu computador
3. Abre Visual Studio Code.
4. Ve al menú **Vista** → **Extensiones** (Ctrl+Shift+X).
5. Haz clic en los tres puntos **...** y selecciona **Instalar desde VSIX...**
6. Selecciona Instalar desde **VSIX...**
7. Busca el archivo **.vsix** en tu computador y haz clic en **Instalar**.
8. Configura en VS Code el api key explicado en la sección **Configuración de de Api Key en VS Code**.

Opción B — Instalar desde el Marketplace

1. Abre Visual Studio Code.
2. Ve a **Vista → Extensiones**.
3. En el buscador escribe: Python Logic Checker.
4. Da clic en el logo de la serpiente verde



5. Haz clic en el botón **Instalar**.
6. Configura el api key , explicado en la sección **Configuración de de Api Key en VS Code**.

¿Cómo obtener tu API Key de OpenRouter?

⚠ IMPORTANTE: La extensión **Python Logic Checker** necesita una clave personal para conectarse. Obtenerla es gratuito y toma menos de 5 minutos.

Paso 1 — Registro

Ve a openrouter.ai y regístrate con tu cuenta de Google o correo electrónico.

Paso 2 — Generación de la Clave

1. En tu perfil, selecciona **API Keys**.
2. Haz clic en **Create Key** y asígnale un nombre (ej: secret-key).
3. **¡Cópiala inmediatamente!** Empieza con sk-or-v1-.... OpenRouter no te la mostrará de nuevo por seguridad.

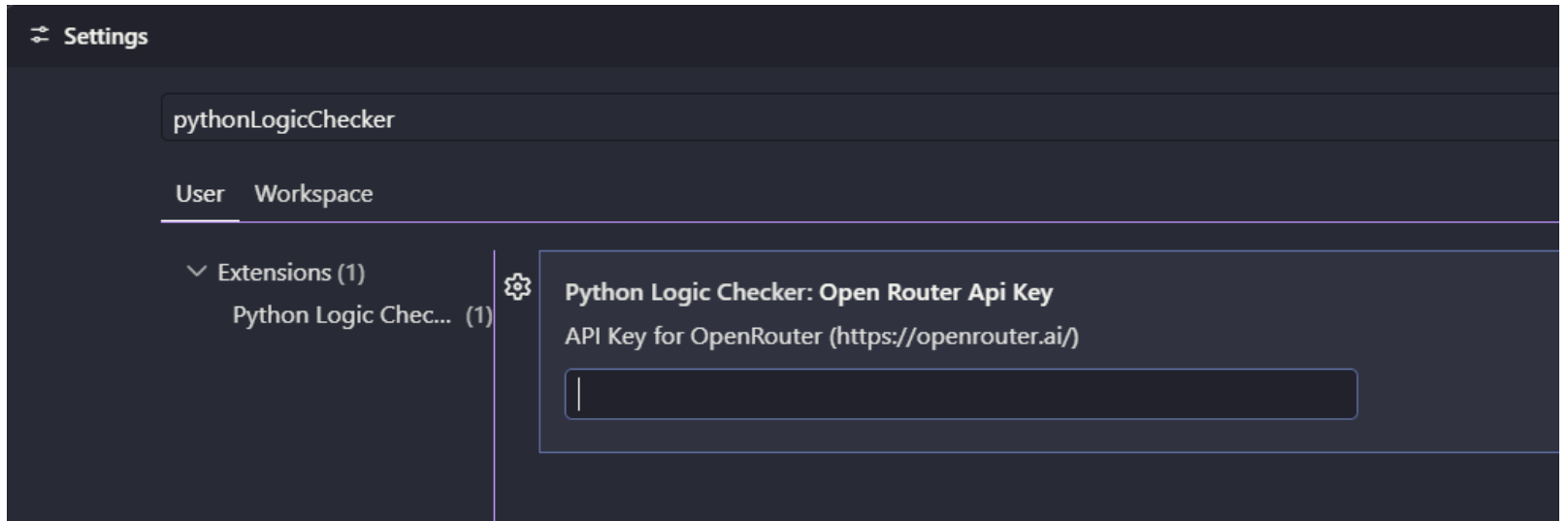
⚠ IMPORTANTE: Trata tu API Key como una contraseña. No la compartas ni la subas a GitHub.

Configuración de la API Key en VS Code para análisis en línea.

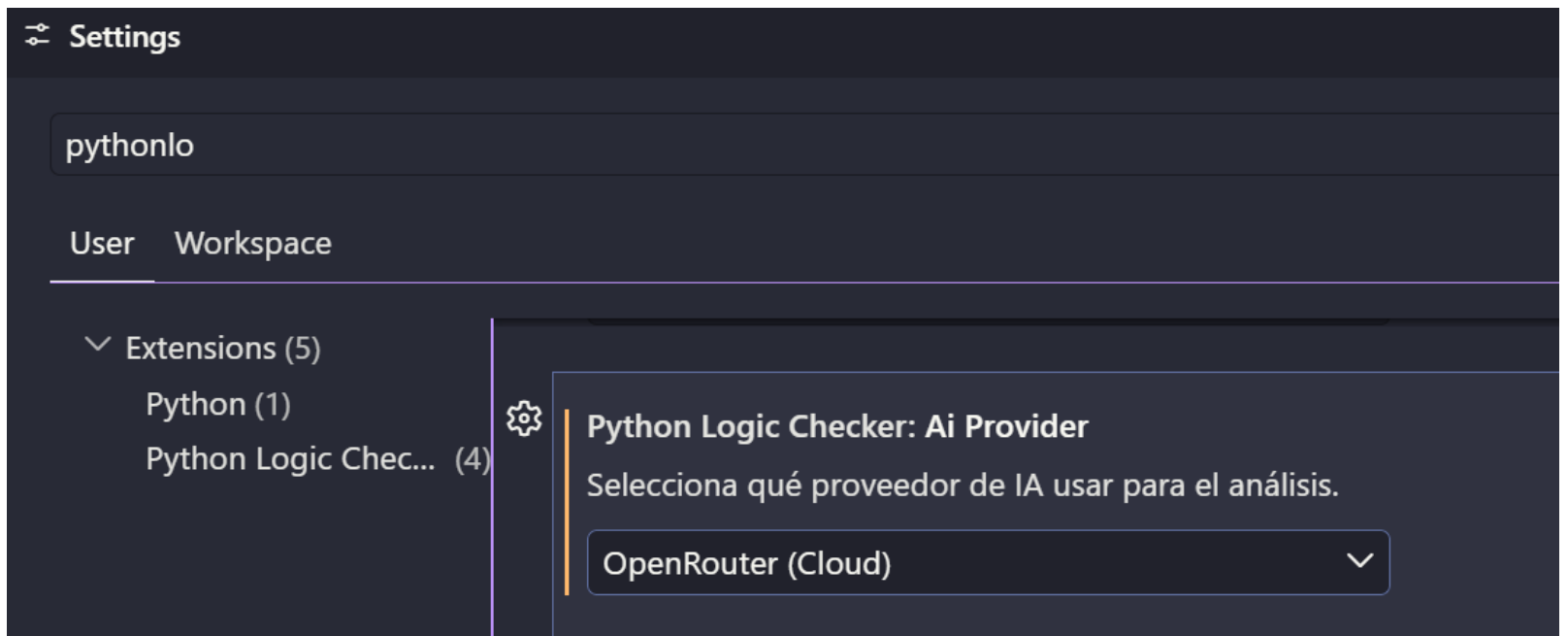
Para el análisis in línea de tu código , sigue los siguientes pasos.

Con tu clave lista, agrégala a la extensión así:

1. En VS Code, ve a **Archivo → Preferencias → Configuración** (o presiona **Ctrl + ,**).
2. En el buscador de configuración escribe: **pythonLogicChecker**.



3. Aparecerá el campo **Open Router Api Key**.
4. Pega tu clave (sk-or-v1-...) en ese campo y cierra la ventana de configuración.
5. Revisa que el Ai provider sea Open Router



6. Reinicia Vs Code.

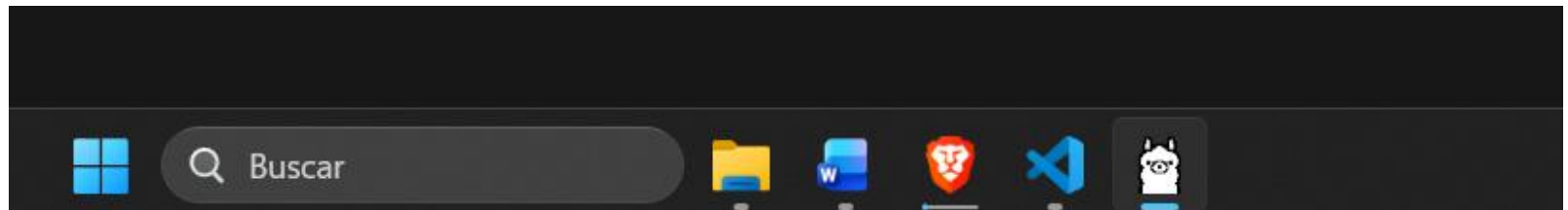
Configuración para el análisis local

⚠ **Nota:** Para el análisis local no es necesario configurar el api key.

Ollama

Descargar Ollama

1. Ingresa a <https://ollama.com/download>.
2. Descarga la versión para tu sistema operativo.
3. Ejecuta el instalador y sigue las instrucciones.
4. Ollama se iniciará automáticamente.
5. Puedes verificar que está se está aparece lista, observando el icono de la llama en tu barra de tareas



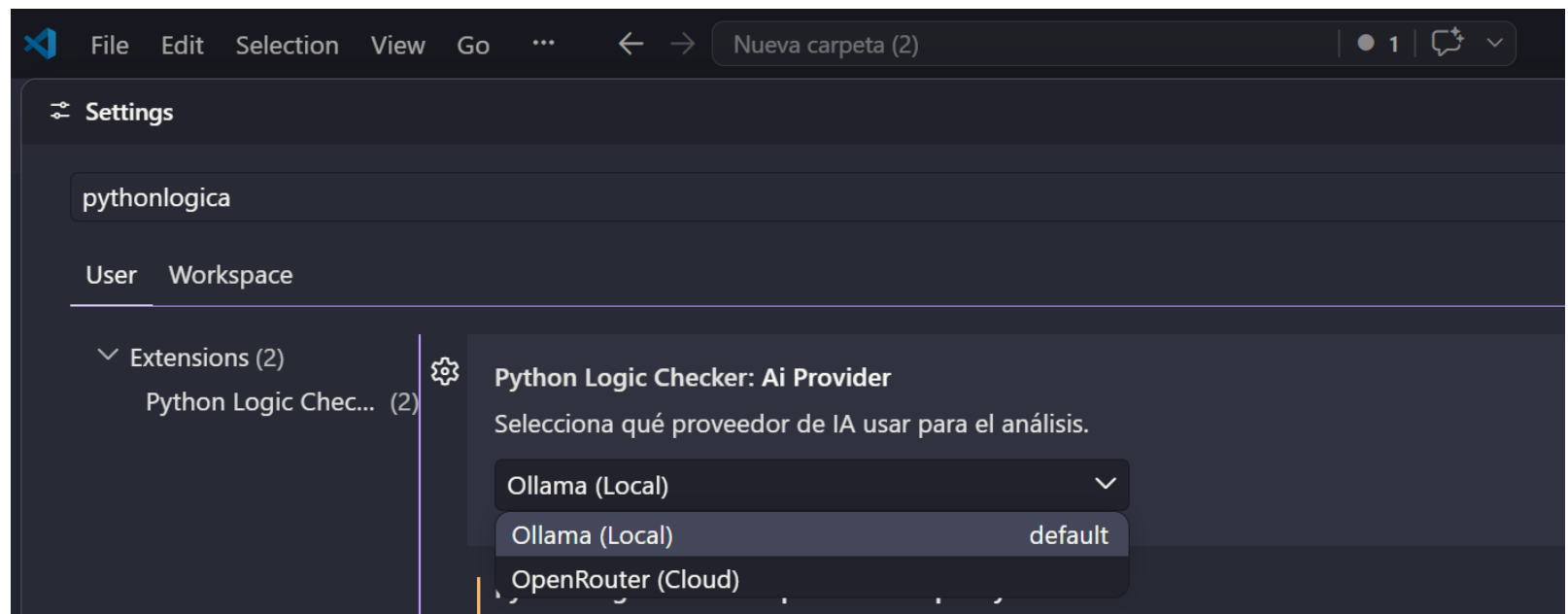
Modelo de Ollama

1. Abre una terminal.
2. Ejecuta el comando **ollama pull phi3**.
3. Espera a que termine la descarga.
4. Verifica que el modelo se haya descargado, ejecuta el comando **ollama list**
5. Aparecerá una lista con los modelos de Ollama descargados.

```
C:\Users\Luz Carine>ollama list
NAME                ID                SIZE             MODIFIED
phi3:latest         4f2222927938     2.2 GB           About an hour ago
```

Configuración del del proveedor de IA

1. En VS Code, ve a Archivo → Preferencias → Configuración (o presiona Ctrl+.).
2. En el buscador de configuración escribe: pythonLogicChecker.
3. Aparecerá el campo Python Logical Cheker: Ai provider .
4. Despliega la lista y selecciona Ollama.
5. Cierra la ventana de configuración.



¿Cómo se usa?

El uso es completamente automático. Solo necesitas seguir estos dos pasos:

1. Abre o crea un archivo Python (cualquier archivo que termine en .py).
2. Escribe tu código y guárdalo con Ctrl+S.

La extensión analizará tu código y, si encuentra errores lógicos, los verás resaltados en verde directamente sobre las líneas problemáticas.

Cómo interpretar los resultados

Evento	Descripción
Resaltado Verde y subrayado azul	Indica que la línea tiene un posible error lógico.
Mensaje al Hover	Pasa el cursor sobre el resaltado para leer la explicación de la IA.
Panel de Problemas	Puedes abrirlo con Vista → Problemas o Presiona Ctrl+Shift+M para ver la lista completa de alertas.

Ejemplos de errores detectados

Ejemplo 1 — Acumulador que no acumula

Este es uno de los errores más frecuentes en principiantes. El código se ve correcto pero el resultado no es el esperado.

Código con error:

```
prueba.py X
prueba.py > ...
1 suma=0
2
3 for i in range(5):
4     suma=i
5
6 print(suma)
```

PROBLEMS 1 DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, **/*.ts, !**/nod...

prueba.py 1

La variable 'suma' debería acumular la 'i' en cada iteración del bucle, pero ... Python Logic Checker [Ln 4, Col 5]

La variable 'suma' debería acumular la 'i' en cada iteración del bucle, pero estás reasignando 'suma' a 'i' en lugar de sumar ('suma = i + suma' o 'suma += i'). Actualmente solo se guarda el último valor de 'i' en 'suma'.

Verás que en VS Code La línea `suma = i` aparecerá resaltada en verde. Al pasar el cursor aparecerá algo como:

"La variable 'suma' debería acumular la 'i' en cada iteración del bucle, pero estás reasignando 'suma' a 'i' en lugar de sumar ('suma=i + suma' o 'suma += i'). Actualmente solo se guarda el último valor de 'i' en 'suma'"

Código corregido:

```
prueba.py > ...
1 suma=0
2
3 for i in range(5):
4     suma +=i
5
6 print(suma)
```

PROBLEMS DEBUG CONSOLE TERMINAL PORTS

No problems have been detected in the workspace.

Ejemplo 2 — Bucle infinito por variable sin modificar

Código con error:

```
prueba.py > ...
1 i=0
2
3 while i <10:
4
5     print("Contando...")
```

PROBLEMS 1 DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, **/*.ts, !**/nod...)

prueba.py 1

El bucle 'while' se ejecutará infinitamente porque la variable 'i' nunca se in... Python Logic Checker [Ln 5, Col 5]

El bucle 'while' se ejecutará infinitamente porque la variable 'i' nunca se incrementa dentro del bucle. Para solucionarlo, debes agregar una línea como 'i = i + 1' dentro del bucle para que la condición de salida ('i < 10') eventualmente sea falsa.

Lo que verás en VS Code: La línea resaltada y le mensaje: El valor de 'i' nunca se incrementa dentro del bucle 'while', lo que significa que la condición 'i < 10' siempre será verdadera y el bucle se ejecutará infinitamente. Para solucionarlo, debes incrementar 'i' en cada iteración, por ejemplo, añadiendo 'i = i + 1' o 'i += 1' al final del bucle.

Código corregido:

```
prueba.py > [i] i
1  i=0
2
3  while i <10:
4
5      print("Contando...")
6
7  i+=1
```

PROBLEMS DEBUG CONSOLE TERMINAL PORTS Filter

No problems have been detected in the workspace.

Ejemplo 3 — Condición que nunca se cumple

Código con error lógico (sin error de sintaxis):

```
prueba.py > ...
1  nota = 75
2
3  if nota > 100:
4
5      print("Aprobado con distinción")
6
7
```

PROBLEMS 1 DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, **/*.ts, !**/*.nod...)

prueba.py 1

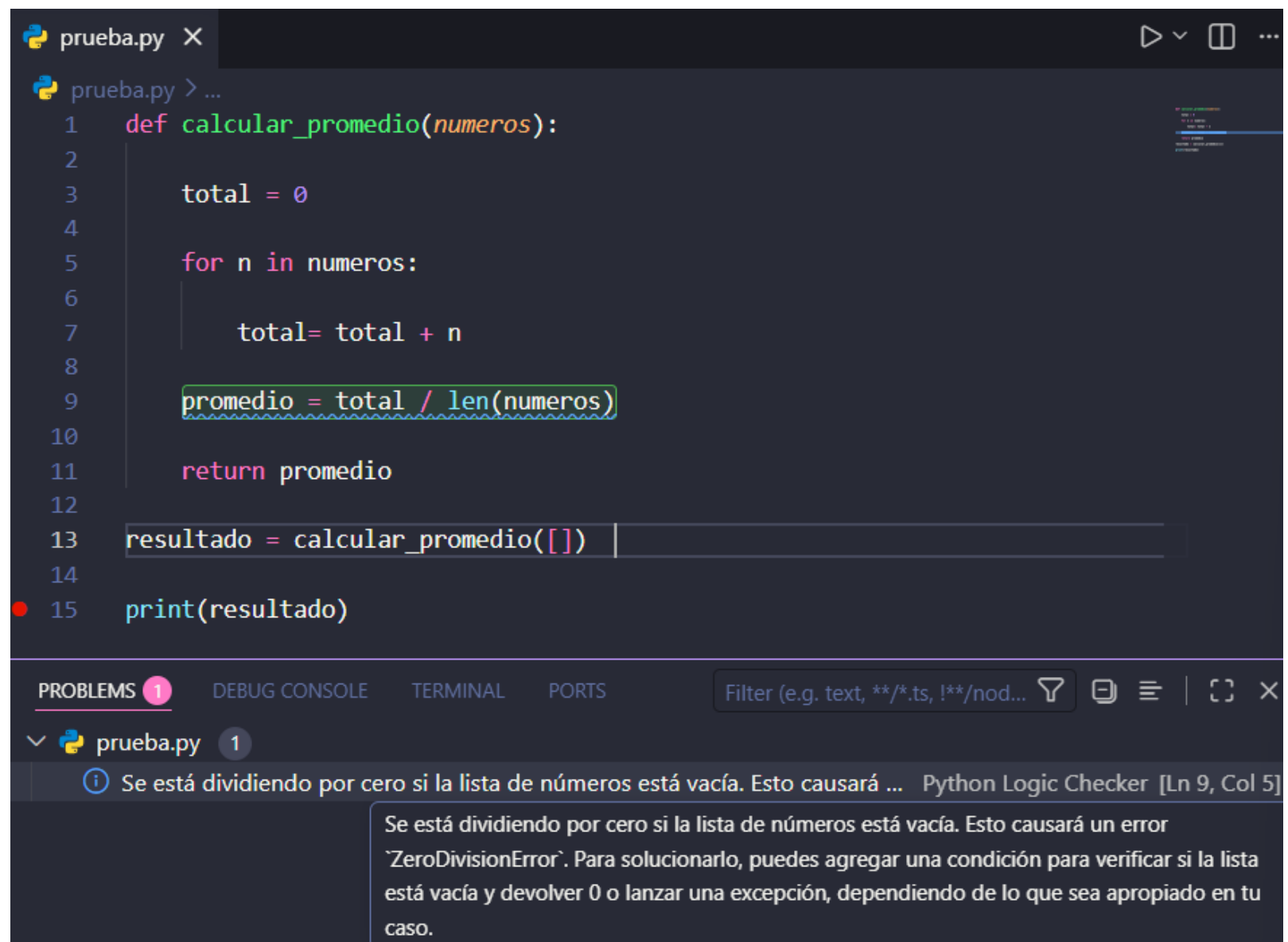
La condición 'nota > 100' nunca se cumplirá en este caso con el valor inici... Python Logic Checker [Ln 3, Col 1]

La condición 'nota > 100' nunca se cumplirá en este caso con el valor inicial de 'nota = 75'. Por lo tanto, el bloque de código dentro del 'if' nunca se ejecutará. Si la intención es verificar si la nota es aprobatoria, deberías usar una condición como 'nota >= 60' (asumiendo que 60 es la nota mínima para aprobar). Considera también agregar un bloque 'else' para el caso en que la nota no es aprobatoria.

Lo que verás en VS Code: La extensión resaltará la condición y explicará que `nota > 100` nunca puede cumplirse y sugiere las posibles correcciones.

Ejemplo 4 — Variable usada antes de ser asignada correctamente

Código con error:



The screenshot shows the VS Code editor with a file named `prueba.py`. The code defines a function `calcular_promedio` that calculates the average of a list of numbers. The function initializes `total = 0`, iterates over the list `numeros` to calculate the sum, and then divides the total by the length of the list. The division line is highlighted with a green box and a wavy underline, indicating a logic error. Below the function, there is a call to `calcular_promedio([])` and a `print` statement. The bottom panel shows the 'PROBLEMS' tab with one error listed: 'Se está dividiendo por cero si la lista de números está vacía. Esto causará ...'. A tooltip provides a detailed explanation of the error and suggests solutions.

```
1 def calcular_promedio(numeros):
2
3     total = 0
4
5     for n in numeros:
6
7         total= total + n
8
9     promedio = total / len(numeros)
10
11     return promedio
12
13 resultado = calcular_promedio([])
14
15 print(resultado)
```

PROBLEMS 1 DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, **/*.ts, !*/nod...    

▼  prueba.py 1

 Se está dividiendo por cero si la lista de números está vacía. Esto causará ... Python Logic Checker [Ln 9, Col 5]

Se está dividiendo por cero si la lista de números está vacía. Esto causará un error `ZeroDivisionError`. Para solucionarlo, puedes agregar una condición para verificar si la lista está vacía y devolver 0 o lanzar una excepción, dependiendo de lo que sea apropiado en tu caso.

Lo que verás en VS Code: La extensión puede alertar que, si `numeros` está vacío, `len(numeros)` será 0 y habrá una división por cero.

Recomendaciones de uso

La extensión realiza una petición al servidor de inteligencia artificial **cada vez que guardas el archivo**.

Para desactivar el autoguardado:

1. Ve a **Archivo → Preferencias → Configuración** (o presiona Ctrl+,).
2. En el buscador escribe: auto save.
3. Busca la opción Files: Auto Save.
4. Cambia el valor a off.

💡 **Buena práctica:** Guarda y analiza tu código cuando hayas terminado de escribir un bloque completo (una función, un ciclo, una sección), no línea por línea.

Preguntas frecuentes

- **¿La extensión revisa errores de escritura?** No. Se enfoca únicamente en errores lógicos.
- **¿Por qué el resaltado es verde y no rojo?** Los errores de sintaxis ya usan el rojo. El verde diferencia los errores lógicos detectados por IA.
- **¿Funciona con cualquier archivo Python?** Sí, con cualquier archivo .py.
- **¿Necesito internet para que funcione?** Sí, el análisis se hace en un servidor de IA, por lo que normalmente necesitas conexión a internet. Sin embargo, si configuras el modo local no necesitas estar conectado, ya que el servidor de Ollama se ejecuta directamente en tu equipo. Esto permite procesar el análisis de código de forma local, utilizando modelos instalados previamente, lo que además mejora la privacidad y reduce la dependencia de servicios externos.
- **¿Cómo desinstalar?**
 1. Ve a **Vista → Extensiones**.
 2. Busca **Python Logic Checker**.
 3. Haz clic en el **ícono de engranaje** ⚙️ y selecciona **Desinstalar**.

Posibles errores de configuración.

⚠️ Error al conectar con la IA

❌ Error al conectar con la IA: fetch failed

Para solucionarlo busca la aplicación de Ollama en tu computador y ejecutala.

⚠ Error de conexión con Open Router

❌ Error al conectar con OpenRouter: OpenRouter API error (504): {"error"...

Cuando no ingresas el api key en la sección de configuración de la extensión, posiblemente obtengas este error. Revisa la sección de configuración de api key para solucionarlo.

Créditos

Extensión desarrollada como parte del Diplomado en Metodologías Ágiles y Tecnologías Avanzadas para el Desarrollo de Software — Universidad de Nariño.